

دانشکده‌ی مهندسی کامپیوتر

هوش مصنوعی و سیستم‌های خبره

پروژه‌ی شبکه‌ی عصبی

دکتر عبدی

پاییز ۱۴۰۵

طراح فاز اول: رامین بوذرپور

طراحان فاز دوم: محمدصالح حیدری، نگار رضائی

فاز اول: آشنایی شهودی با شبکه‌ی عصبی

طراح فاز اول: رامین بوذرپور

۱ هدف ورکشاپ

این ورکشاپ شبکه عصبی را از حالت فرمول و کاغذ خارج می‌کند و نشان می‌دهد هر جزء آن در عمل چه می‌کند. چهار تسک دارد و هر چهار در مرورگر اجرا می‌شوند؛ نیازی به نصب برنامه یا نوشتن کد نیست. کار شما در هر تسک یکسان است: یک پارامتر را تغییر دهید، نتیجه را مشاهده کنید و دلیل آن را بنویسید.

این ورکشاپ به تصویر و شبکه‌های کانولوشنی نمی‌پردازد؛ تمرکز آن فقط روی ساختار شبکه و عوامل مؤثر بر آموزش آن است.

در پایان باید بتوانید:

- بگویید یک شبکه از چه اجزایی ساخته شده و هر جزء چه کاری می‌کند.
- پیش‌بینی کنید تغییر activation، تعداد نورون‌ها، learning rate یا regularization چه اثری می‌گذارد.
- از روی نمودار loss تشخیص دهید مدل underfit شده یا overfit.
- مفهوم vanishing gradient را با زبان خودتان توضیح دهید.

۲ نقشه‌ی تسک‌ها

بهتر است به همین ترتیب جلو بروید. تسک اول ساده‌ترین است و آخرین‌ها به گرادیان می‌رسند.

تسک	سایت	چه چیزی را می‌بینید
۱	TensorFlow Playground	اثر activation، learning rate، regularization و feature روی یک شبکه‌ی کوچک
۲	MLU-Explain	فرق underfit و overfit و دلیل جدا کردن داده‌ی validation
۳	Backprop Explainer	گرادیان لایه‌ها و مشکل vanishing gradient
۴	همه‌ی سایت‌ها	یک گزارش کوتاه که همه را به هم وصل می‌کند

۳ نحوه‌ی کار و تحویل

هر تسک چند کار مشخص دارد. کار را انجام دهید، از نتیجه اسکرین‌شات بگیرید یا عددش را یادداشت کنید و به سؤالی که دنبالش آمده جواب بدهید. جواب دو تا چهار جمله باشد و با زبان خودتان.

یک قاعده از همه مهم‌تر است: پیش از هر تغییر، پیش‌بینی خودتان را بنویسید («فکر می‌کنم loss نوسان می‌کند، چون...»)، بعد تغییر را اعمال کنید. اگر پیش‌بینی‌تان درست نبود، همان مغایرت را توضیح دهید؛ دقیقاً همین جاست که مطلب واقعی یاد گرفته می‌شود.

تحویل یک فایل PDF است با اسکرین‌شات‌ها و جواب‌ها، به ترتیب تسک‌ها. شماره‌ی هر کار را کنار جوابش بنویسید (مثلاً «تسک ۱، کار ۳») تا بررسی راحت‌تر شود.

۴ تسک ۱: TensorFlow Playground

آدرس: playground.tensorflow.org.

رنگ نارنجی یعنی مقدار منفی و آبی یعنی مثبت. خطی که دو نورون را وصل می‌کند هرچه ضخیم‌تر باشد، وزنش بزرگ‌تر است، و اگر ماوس را روی یک نورون نگه دارید خروجی‌اش را بزرگ‌تر می‌بینید. بالای صفحه هم epoch، learning rate، activation، regularization و نوع مسئله را تغییر می‌دهید. تغییر هر کدام از این مقادیر بازگشت‌پذیر است، پس برای آزمایش محدودیتی ندارید.

۱. دیتاست Circle را انتخاب کنید، فقط x_1 و x_2 را ورودی بگذارید و یک لایه‌ی مخفی با چهار نورون بسازید. چهار بار آموزش بدهید، هر بار با یکی از activationهای Linear، ReLU، Tanh و Sigmoid و به همان تعداد epoch. test loss و شکل مرز تصمیم را هر بار ثبت کنید. چرا Linear، با هر تعداد نورون و لایه، نمی‌تواند یک دایره را جدا کند؟ (راهنما: ترکیب چند تابع خطی چه نوع تابعی می‌شود؟)

۲. این بار activation را روی Tanh نگه دارید و learning rate را از تا بالا ببرید (منوی بازشو همه‌ی مقدارها را دارد). برای هر مقدار بنویسید loss کند پایین می‌آید، پایدار است، نوسان می‌کند یا واگرا می‌شود. بهترین بازه کجا بود؟ چرا مقدار خیلی بزرگ همه‌چیز را خراب می‌کند و مقدار خیلی کوچک تقریباً کاری نمی‌کند؟

۳. حالا دیتایی بسازید که نویز دارد: noise را بالا ببرید و سهم داده‌ی train را کم کنید. یک شبکه‌ی بزرگ (مثلاً سه لایه‌ی شش تا هشت‌نورونی) بسازید و آن‌قدر آموزش بدهید که train loss خیلی پایین‌تر از test loss برود. از نمودار اسکرین‌شات بگیرید. بعد L2 و سپس L1 را با چند regularization rate امتحان کنید و ببینید ضخامت خط‌های وزن چه می‌شود. شکاف بین train loss و test loss یعنی چه؟ در حالت L1 نگاه کنید که آیا وزن‌هایی کاملاً به صفر می‌رسند (sparsity)؛ در حالت L2 نگاه کنید که آیا همه‌ی وزن‌ها به‌طور یکنواخت کوچک‌تر می‌شوند (shrinkage). این تفاوت چه ربطی به overfitting و ساده‌تر شدن مرز تصمیم دارد؟

۴. دیتای دایره‌ای را یک بار با x_1 و x_2 و یک لایه‌ی مخفی حل کنید. بار دوم همه‌ی لایه‌های مخفی را بردارید و به‌جایش x_1^2 و x_2^2 را هم به ورودی‌ها اضافه کنید. دو نتیجه را کنار هم بگذارید. افزودن این دو feature در واقع مسئله را از یک مسئله‌ی غیرخطی در فضای اصلی به یک مسئله‌ی خطی در فضای ویژگی‌های جدید تبدیل کرده است؛ همین تبدیل چه کاری را به‌جای عمق شبکه انجام می‌دهد؟ چرا بدون هیچ لایه‌ی مخفی هم مرز دایره‌ای ممکن شد؟

۵ تسک ۲: MLU-Explain

مقاله‌های این سایت صفحه‌هایی هستند که با اسکرول کردن جلو می‌روند: هرچه پایین‌تر بروید نمودارها عوض می‌شوند و وسط متن چند بخش تعاملی هست. صفحه‌ی اصلی (mlu-explain.github.io) فقط فهرست مقاله‌هاست و گم شدن در آن آسان است، پس از لینک‌هایی که پایین آمده شروع کنید.

۱. مقاله‌ی Bias-Variance را از اول بخوانید. روی مثال‌های تعاملی LOESS و KNN پیچیدگی مدل (مثلاً تعداد همسایه‌ها) را از خیلی ساده تا خیلی پیچیده ببرید و از سه حالت اسکرین‌شات بگیرید: underfit، حالت مناسب و overfit. هر حالت را با bias و variance توضیح دهید. چرا نمی‌شود هر دو را همزمان صفر کرد؟

۲. به تسک ۱ برگردید. کدام تنظیم‌های Playground مدل را به سمت underfit می‌بردند و کدام به سمت overfit؟ یک جدول دوستونه بسازید که در یک ستونش مفهوم باشد (underfit و overfit) و در ستون دیگر تنظیم مشابهش در Playground.

۳. مقاله‌ی Train, Test, and Validation Sets را بخوانید و با مدل زنده‌ی خودش ببینید اگر فقط روی داده‌ی train ارزیابی کنید چه می‌شود. یک مثال پیدا کنید که train عالی و test بد باشد. چرا به یک مجموعه‌ی validation جدا نیاز داریم و داده‌ی test چه موقع باید دست بخورد؟

۶ تسک ۳: Backprop Explainer

آدرس: xnought.github.io/backprop-explainer.

اگر کنترلی با اسمی که پایین آمده در ابزار نبود، نزدیک‌ترین کنترل را امتحان کنید و در گزارش بنویسید چه کردید.

۱. یک شبکه‌ی کوچک را روی دیتای دوبعدی آموزش بدهید و ببینید فضای ورودی با هر epoch چطور تغییر شکل می‌دهد. از قبل و بعد آموزش اسکرین‌شات بگیرید. هر لایه با داده چه می‌کند و آموزش چه چیزی را در این تغییر عوض می‌کند؟

۲. عمق شبکه را زیاد کنید (مثلاً به پنج یا شش لایه برسانید) و گرادیان لایه‌های اول را با گرادیان لایه‌های آخر مقایسه کنید. توجه کنید که این ابزار activation لایه‌های مخفی را روی ReLU ثابت نگه می‌دارد و سوئیچی برای عوض کردنش ندارد؛ اگر می‌خواهید اثر خودِ activation روی گرادیان را هم ببینید، به تسک ۱ برگردید و در Playground بین Sigmoid، Tanh و ReLU سوئیچ کنید. همین‌جا فقط اثر عمق را بررسی کنید: vanishing gradient را با زبان خودتان توضیح دهید و بگویید چرا هرچه شبکه عمیق‌تر می‌شود این مشکل بیشتر خودش را نشان می‌دهد.

۷ تسک ۴: جمع‌بندی

این بخش همه‌ی چیزهایی را که دیده‌اید کنار هم می‌گذارد. گزارشش از یک صفحه بیشتر نشود.

۱. یک دیتاست دلخواه از Playground انتخاب کنید (Circle نباشد) و بهترین تنظیمی را که می‌توانید پیدا کنید: معماری، activation، learning rate و regularization. برای هر انتخاب یک جمله دلیل بنویسید و اگر به یکی از کارهای قبلی مربوط است، شماره‌اش را بیاورید (مثلاً «طبق تسک ۱، کار ۲، learning rate را گذاشتم چون...»).

۲. جدول زیر را پر کنید. برای هر ردیف دو جمله کافی است؛ تا جایی که مرتبط است، پاسخ را به bias/variance یا underfit/overfit وصل کنید.

عامل	اثر افزایش	اثر کاهش
تعداد نورون‌ها و لایه‌ها		
learning rate		
regularization rate		
نویز داده		
حجم داده‌ی train		

فاز دوم: پیاده‌سازی شبکه‌ی عصبی پرسپترون چندلایه (MLP) با NumPy

طراحان فاز دوم: محمدصالح حیدری، نگار رضائی

۱ مقدمه و اهداف پروژه

در این پروژه، شما یک چارچوب کامل شبکه عصبی پرسپترون چندلایه (Multi-Layer Perceptron - MLP) را از پایه و صرفاً با بهره‌گیری از کتابخانه NumPy پیاده‌سازی خواهید کرد.

در این پروژه، محاسبات به شکل ماتریسی و برداری انجام می‌شوند تا کدها هم سریع‌تر اجرا شوند و هم خواناتر باشند. بنابراین، به‌جای استفاده از حلقه‌های تودرتو (Nested Loops) روی تک‌تک نمونه‌ها (که سربار محاسباتی قابل‌توجهی دارد)، ویژگی‌ها یا نورون‌ها، فرآیندهای پیش‌پردازش، انتشار رو به جلو، محاسبهٔ تابع هزینه، منظم‌سازی L_2، پس‌انتشار خطا و معیارهای ارزیابی به شکل عملیات ماتریسی و برداری پیاده‌سازی می‌شوند. استفاده از کتابخانه‌های سطح بالای یادگیری عمیق (مانند PyTorch، TensorFlow، Keras) یا مدل‌های آماده Scikit-Learn در این پروژه مجاز نیست.

این پروژه از سه بخش اصلی تشکیل شده است:

۱. **بخش اول – بررسی و آماده‌سازی دیتاست (پیش‌پردازش):** پاک‌سازی، مدیریت داده‌های گمشده و پرت، و تبدیل ویژگی‌ها بدون نشت اطلاعات.

۲. **بخش دوم – پیاده‌سازی شبکه عصبی MLP برای دسته‌بندی دودویی:** طراحی و آموزش یک شبکه عصبی ۴ لایه برای پیش‌بینی سطح درآمد.

۳. **بخش سوم – تحلیل:** اجرای آزمایش‌های تجربی و تحلیلی جهت بررسی رفتار الگوریتم‌های بهینه‌سازی، توابع فعال‌سازی، نرخ یادگیری، منظم‌سازی، مقداردهی اولیه وزن‌ها و ماتریس درهم‌ریختگی.

۲ بخش اول: بررسی و آماده‌سازی دیتاست

2.1 معرفی دیتاست

در این پروژه از دیتاست `adult_full_merged.csv` برای پیش‌بینی سطح درآمد افراد بر اساس ویژگی‌های جمعیت‌شناختی، آموزشی و شغلی استفاده می‌شود. هر سطر از دیتاست نمایانگر یک نمونه و هر ستون

نشان‌دهنده یک ویژگی از آن نمونه یا متغیر هدف است. متغیر هدف، ستون `income` است که دو حالت دارد: `>50K` (درآمد سالانهٔ بیش از ۵۰ هزار دلار) و `<=50K` (درآمد سالانهٔ کمتر یا مساوی ۵۰ هزار دلار).

این دیتاست عمداً به‌گونه‌ای آماده شده است که شامل برخی مشکلات کیفی در داده‌ها باشد. بنابراین، پیش از استفاده از آن برای آموزش شبکه عصبی، لازم است وضعیت داده‌ها را بررسی کرده و مراحل مناسب پیش‌پردازش را انجام دهید. هدف این مرحله، شناسایی و مدیریت مشکلات داده‌ای و تولید یک دیتاست قابل‌اعتماد برای مراحل بعدی پروژه است.

2.2 وظایف دانشجو در بخش پیش‌پردازش

- **انتخاب ویژگی‌ها:** همه ستون‌ها الزاماً برای پیش‌بینی مفید نیستند. ارتباط ویژگی‌ها با مسئله و میزان اطلاعات مفید آن‌ها را بررسی کنید و ویژگی‌های غیرضروری یا نامرتبط را شناسایی و حذف کنید.

- **یکسان‌سازی مقادیر دسته‌ای:** ممکن است یک مقدار به شکل‌های مختلف (حروف بزرگ/کوچک متفاوت، فاصله‌ی اضافی، شکل مختصر) نوشته شده باشد. اگر این ناسازگاری‌ها اصلاح نشوند، در مرحله‌ی رمزگذاری دسته‌های تکراری و بی‌معنا ایجاد می‌شوند. برای مثال حالت فضای خالی، `?`، `NA`، `unknown` همگی مربوط به داده گمشده می‌باشد. مثال دیگر `united states`، `US`، `USA` هرسه یکسان هستند (و همچنین `M`، `male` و `MALE` در ستون جنسیت).

- **رکوردهای تکراری:** بررسی کنید آیا رکورد تکراری در داده وجود دارد؛ حضور آن‌ها باعث می‌شود مدل روی برخی نمونه‌ها بیش از حد آموزش ببیند و نتیجه‌ی ارزیابی خوش‌بینانه شود. پس از یکسان‌سازی رشته‌ها، رکوردهای تکراری را شمارش و حذف نمایید.

- **مقادیر گمشده:** نسبت مقادیر خالی در هر ستون را محاسبه کنید و بر اساس نوع ستون و میزان گمشدگی، راهبرد مناسبی (حذف، جایگزینی با مقدار مرکزی مانند میانه برای داده‌های عددی و مد برای داده‌های دسته‌ای، یا دسته‌ی مشخص) انتخاب کنید. دقت کنید این عملیات نباید باعث نشت اطلاعات از داده‌ی آزمون به آموزش شود.

- **مقادیر پرت:** برخی مقادیر عددی ممکن است خارج از بازه‌ی منطقی باشند (مانند سن منفی یا ۲۰۰ سال، ساعات کاری ۳۰۰ ساعت در هفته، یا سود سرمایه ۱۰ میلیون دلاری). این مقادیر می‌توانند بر نرمال‌سازی و آموزش شبکه اثر منفی بگذارند. راهبرد مناسب (محدودکردن، حذف، یا روش‌های مقاوم‌تر) و بازه‌ی منطقی هر ستون را خودتان بررسی، انتخاب و اعمال کنید.

- **رمزگذاری و مقیاس‌بندی** متغیرهای دسته‌ای را با روش مناسب (با توجه به ترتیبی بودن یا نبودن) به شکل عددی تبدیل کنید و ویژگی‌های عددی را با روش مناسبی مقیاس‌بندی نمایید. پارامترهای مقیاس‌بندی باید فقط با استفاده از داده‌ی آموزش محاسبه شوند تا از نشت اطلاعات جلوگیری شود:

۱. **رمزگذاری متغیر هدف (Target Encoding):** تبدیل مقادیر `50k=>` به عدد 0 و `50k>` به عدد 1.

۲. **رمزگذاری ترتیبی (Ordinal Encoding):** تبدیل ستون `education` بر اساس سلسله‌مراتب طبیعی تحصیلات (از `preschool` برابر با 1 تا `doctorate` برابر با 18).

۳. **رمزگذاری دودویی بردار (Vectorized One-Hot Encoding):** برای ویژگی‌های اسمی (Nominal) مانند `workclass`، `marital-status`، `occupation`، `relationship`، `race`، `sex` و `native-country`، به‌جای نوشتن حلقه روی مقادیر یکتا، از مقایسه تساوی برداری در NumPy استفاده کنید. اگر بردار ستونی مقادیر ویژگی را با $\mathbf{v} \in \mathbb{R}^{N\times 1}$ و بردار سطری مقادیر یکتا را با $\mathbf{u}^T \in \mathbb{R}^{1\times K}$ نمایش دهیم، ماتریس دودویی یک‌باره از رابطه زیر حاصل می‌شود:

$$B = (\mathbf{v} == \mathbf{u}^T) \in \{0,1\}^{N\times K} \quad (۱)$$

۴. **افراز آموزش/آزمون و استانداردسازی (Z-Score Standardization):** ابتدا داده‌ها را به دو بخش آموزش (80%) و آزمون (20%) تقسیم کرده و ماتریس‌ها را ترانهاده کنید تا هر ستون نمایانگر یک نمونه باشد: $X_{\text{train}} \in \mathbb{R}^{I\times m_{\text{train}}}$ و $X_{\text{test}} \in \mathbb{R}^{I\times m_{\text{test}}}$. سپس برای d سطر متناظر با ویژگی‌های عددی، بردار میانگین $\boldsymbol{\mu} \in \mathbb{R}^{d\times 1}$ و بردار انحراف معیار $\boldsymbol{\sigma} \in \mathbb{R}^{d\times 1}$ را صرفاً روی داده‌های آموزش محاسبه کرده و هر دو ماتریس آموزش و آزمون را استاندارد نمایید:

$$Z_{\text{num}} = \frac{X_{\text{num}} - \boldsymbol{\mu}\mathbf{1}_m^T}{\boldsymbol{\sigma}\mathbf{1}_m^T} \quad (۲)$$

۳ بخش دوم: پیاده‌سازی شبکه عصبی MLP

در این بخش، یک شبکه عصبی تمام‌متصل ۴ لایه (4-Layer MLP) برای دسته‌بندی دودویی سطح درآمد طراحی و پیاده‌سازی می‌کنیم.

3.1 معماری شبکه و ابعاد ماتریس‌ها

معماری شبکه به‌صورت زیر تعریف می‌شود:

$$\texttt{Input } (X) \rightarrow \texttt{[FC1} \rightarrow \texttt{ReLU]} \rightarrow \texttt{[FC2} \rightarrow \texttt{ReLU]} \rightarrow \texttt{[FC3} \rightarrow \texttt{ReLU]} \rightarrow \texttt{[FC4} \rightarrow \texttt{Sigmoid]} \rightarrow \texttt{Output } (A^{[4]})$$

برای یک دسته (Batch) شامل m نمونه که در ستون‌های ماتریس $X \in \mathbb{R}^{I\times m}$ قرار گرفته‌اند، ابعاد پارامترها عبارت‌اند از:

- **لایهٔ ورودی:** $A^{[0]} = X \in \mathbb{R}^{I \times m}$ (که در آن $I \approx 90$ تعداد کل ویژگی‌ها پس از پیش‌پردازش است).
- **لایهٔ پنهان اول ($H_1 = 64$):** $W^{[1]} \in \mathbb{R}^{64 \times I}$ و بردار بایاس $b^{[1]} \in \mathbb{R}^{64 \times 1}$.
- **لایهٔ پنهان دوم ($H_2 = 32$):** $W^{[2]} \in \mathbb{R}^{32 \times 64}$ و بردار بایاس $b^{[2]} \in \mathbb{R}^{32 \times 1}$.
- **لایهٔ پنهان سوم ($H_3 = 16$):** $W^{[3]} \in \mathbb{R}^{16 \times 32}$ و بردار بایاس $b^{[3]} \in \mathbb{R}^{16 \times 1}$.
- **لایهٔ خروجی ($O = 1$):** $W^{[4]} \in \mathbb{R}^{1 \times 16}$ و بردار بایاس $b^{[4]} \in \mathbb{R}^{1 \times 1}$.

3.2 مقداردهی اولیهٔ وزن‌ها (روش He Initialization)

تابع مقداردهی اولیهٔ وزن‌ها به روش He Initialization در نوت‌بوک در اختیار شما قرار گرفته است تا واریانس خروجی لایه‌های دارای فعال‌ساز ReLU حفظ شود:

$$W^{[l]} \sim \mathcal{N}\left(0, \sqrt{\frac{2}{n^{[l-1]}}}\right), \qquad b^{[l]} = \mathbf{0} \tag{۳}$$

3.3 وظیفهٔ ۲-۳: پیاده‌سازی توابع فعال‌سازی و مشتق آن‌ها

توابع فعال‌سازی ReLU (برای لایه‌های پنهان) و Sigmoid (برای لایهٔ خروجی و آزمایش‌های بخش سوم) را به همراه مشتق آن‌ها به‌صورت درایه‌ای (Element-wise) پیاده‌سازی کنید:

$$\mathsf{ReLU}(Z) = \max(0, Z), \qquad \frac{\partial \mathsf{ReLU}(Z)}{\partial Z} = \mathbb{I}(Z > 0) \tag{۴}$$

$$\sigma(Z) = \frac{1}{1 + e^{-Z}}, \qquad \sigma'(Z) = \sigma(Z) \odot \big(1 - \sigma(Z)\big) \tag{۵}$$

نکتهٔ محاسباتی: برای جلوگیری از سرریز عددی (Overflow) در تابع نمایی، مقادیر ورودی Z در تابع سیگموید را پیش از محاسبه به بازه $[-500, 500]$ محدود (`np.clip`) نمایید.

3.4 وظیفه ۲-۴: انتشار رو به جلو (Forward Pass)

در یک پیاده‌سازی ساده (و غیربرداری)، محاسبهٔ تبدیل خطی یک لایه مستلزم سه حلقه تودرتو روی نمونه‌ها (k)، نورون‌های خروجی (i) و نورون‌های ورودی (j) است:

$$Z^{[l]}_{i,k} = \sum_{j=1}^{n^{[l-1]}} W^{[l]}_{i,j} A^{[l-1]}_{j,k} + b^{[l]}_{i,1} \tag{۶}$$

در پیاده‌سازی ماتریسی، کل ماتریس پیش‌فعال‌سازی $Z^{[l]} \in \mathbb{R}^{n^{[l]} \times m}$ در یک گام با استفاده از ضرب ماتریسی و پخش بایاس محاسبه می‌شود:

$$Z^{[l]} = W^{[l]} A^{[l-1]} + b^{[l]} \mathbf{1}_m^T \tag{۷}$$

که در آن $\mathbf{1}_m \in \mathbb{R}^{m \times 1}$ بردار ستونی یک است.

۱. تابع `affine_forward_matrix(A_prev, W, b)` را با استفاده از ضرب ماتریسی پیاده‌سازی کرده و سرعت اجرای آن را با نسخه مبتنی بر حلقه مقایسه کنید.

۲. تابع `feed_forward(X, params)` را برای شبکهٔ ۴ لایه پیاده‌سازی نمایید به طوری که لایه‌های ۱ تا ۳ از فعال‌ساز ReLU و لایه ۴ از فعال‌ساز Sigmoid استفاده کنند و مقادیر میانی در لیست `cache` ذخیره شوند.

3.5 وظیفهٔ ۲-۵: تابع هزینه آنتروپی متقاطع دودویی (BCE)

تابع هزینهٔ آنتروپی متقاطع دودویی (Binary Cross-Entropy - BCE) روی m نمونه به‌صورت زیر تعریف می‌شود:

$$L_{\text{BCE}} = -\frac{1}{m} \sum_{i=1}^{m} \left(y_d^{(i)} \log(\hat{y}^{(i)}) + (1 - y_d^{(i)}) \log(1 - \hat{y}^{(i)}) \right) \tag{۸}$$

با قرار دادن برچسب‌های حقیقی $y_d \in \mathbb{R}^{1 \times m}$ و احتمالات پیش‌بینی‌شده $\hat{y} = A^{[4]} \in \mathbb{R}^{1 \times m}$ به‌صورت بردارهای سطری، مجموع حاصل‌ضرب‌های درایه‌ای دو بردار را می‌توان بدون استفاده از حلقه یا تابع `np.sum` و مستقیماً با ضرب ماتریسی بردار سطری در ترانهادهٔ بردار دیگر محاسبه کرد:

$$L_{\text{BCE}} = -\frac{1}{m} \Big(y_d \big(\log(\hat{y}) \big)^T + (1 - y_d) \big(\log(1 - \hat{y}) \big)^T \Big) \tag{۹}$$

3.6 وظیفهٔ ۲-۶: منظم‌سازی L_2

جریمهٔ منظم‌سازی L_2 متناسب با مجموع مربعات تمام درایه‌های ماتریس‌های وزن است:

$$L_{\text{reg}} = \frac{\lambda}{2m} \sum_{l=1}^{4} \|W^{[l]}\|_F^2 \tag{۱۰}$$

برای هر ماتریس حقیقی $W \in \mathbb{R}^{r \times c}$، مجموع مربعات تمام درایه‌های ماتریس برابر است با اثر (Trace) حاصل‌ضرب ماتریس در ترانهاده خودش (ماتریس WW^T):

$$\|W^{[l]}\|_F^2 = \sum_{i=1}^{r} \sum_{j=1}^{c} \left(W_{i,j}^{[l]} \right)^2 = \operatorname{tr} \left(W^{[l]} (W^{[l]})^T \right) \tag{۱۱}$$

تابع `regularization_L2` را با استفاده از `np.trace(W @ W.T)` پیاده‌سازی نمایید.

3.7 وظیفهٔ ۲-۷: پس‌انتشار خطا (Backpropagation)

گرادیان‌های شبکهٔ ۴ لایه را در تابع `Backpropagation` با عملیات ماتریسی محاسبه کنید:

۱. **گرادیان لایهٔ خروجی ($l = 4$، ترکیب سیگموید و BCE):**

$$dZ^{[4]} = \frac{\partial L}{\partial Z^{[4]}} = \frac{1}{m}\left(A^{[4]} - y_d\right) \in \mathbb{R}^{1\times m} \tag{۱۲}$$

۲. **گرادیان وزن‌ها و بایاس‌ها برای هر لایهٔ $l \in \{4,3,2,1\}$:** جمع زدن ستون‌های ماتریس $dZ^{[l]}$ را می‌توان به‌جای استفاده از `np.sum`، با ضرب ماتریسی $dZ^{[l]}$ در بردار ستونی یک $\mathbf{1}_m \in \mathbb{R}^{m\times 1}$ انجام داد:

$$dW^{[l]} = dZ^{[l]}\left(A^{[l-1]}\right)^T + \frac{\lambda}{m}W^{[l]}, \qquad db^{[l]} = dZ^{[l]}\mathbf{1}_m \tag{۱۳}$$

۳. **انتشار خطا به لایه‌های پنهان قبلی ($l \in \{3,2,1\}$):**

$$dA^{[l]} = \left(W^{[l+1]}\right)^T dZ^{[l+1]}, \qquad dZ^{[l]} = dA^{[l]} \odot g'\left(Z^{[l]}\right) \tag{۱۴}$$

3.8 وظیفهٔ ۲-۸: حلقهٔ آموزش مدل (Batch Gradient Descent)

در تابع `training_clf`، مراحل انتشار رو به جلو، محاسبهٔ هزینهٔ کل ($L_{\text{total}} = L_{\text{BCE}} + L_{\text{reg}}$)، پس‌انتشار خطا و به‌روزرسانی پارامترها با نرخ یادگیری η را ترکیب کرده و مدل را روی دیتاست آموزش دهید:

$$W^{[l]} \leftarrow W^{[l]} - \eta\, dW^{[l]}, \qquad b^{[l]} \leftarrow b^{[l]} - \eta\, db^{[l]} \tag{۱۵}$$

۴ بخش سوم: تحلیل

در این بخش، با استفاده از مدل آموزش‌دیده روی دیتاست Adult Census Income، سه آزمایش تحلیلی زیر را اجرا کرده و به سؤالات تشریحی در نوت‌بوک پاسخ دهید:

4.1 آزمایش ۳-۱: مقایسهٔ Batch GD، Mini-Batch GD و SGD

تابع `training_clf_minibatch` را پیاده‌سازی کنید که در هر دور (Epoch)، پس از بُر زدن داده‌ها (Shuffling)، ماتریس X را به زیرماتریس‌هایی با اندازه دسته B (`batch_size`) تقسیم کند.

- با قرار دادن $B = m$ (گرادیان کاهشی دسته‌ای کامل)، $B = 128$ (گرادیان کاهشی دسته‌ای کوچک) و $B = 1$ (گرادیان کاهشی تصادفی خالص SGD)، مدل را به مدت ۱۰ دور روی ۲۰۰۰ نمونه اول آموزش دهید.
- **تحلیل:** نمودار کاهش هزینه برحسب دور و همچنین **زمان اجرای واقعی (Wall-Clock Time)** هر سه روش را مقایسه کنید.

4.2 آزمایش ۳-۲: بررسی توابع فعال‌سازی لایه‌های پنهان (ReLU در مقایسه با Sigmoid)

توابع `feed_forward_flexible`، `Backpropagation_flexible` و `training_flexible` را به گونه‌ای تکمیل کنید که نوع تابع فعال‌سازی لایه‌های پنهان ۱ تا ۳ (`'relu'` یا `'sigmoid'`) را به عنوان ورودی دریافت کنند (لایه ۴ همواره Sigmoid باقی می‌ماند).

- دو مدل را به مدت ۳۰۰ تکرار آموزش داده و منحنی هزینه آن‌ها را در یک نمودار رسم کنید.

- **تحلیل:** کدام تابع فعال‌سازی سریع‌تر همگرا می‌شود؟ پدیده **محو شدن گرادیان (Vanishing Gradient)** در شبکه سیگموید را با توجه به کران بالای مشتق سیگموید ($\max_z \sigma'(z) = 0.25$) و ضرب متوالی ماتریس‌ها در طول پس‌انتشار تشریح نمایید.

4.3 آزمایش ۳-۳: تحلیل نرخ یادگیری (η)

مدل را با سه نرخ یادگیری متفاوت به مدت ۱۵۰ تکرار آموزش دهید: نرخ یادگیری بالا ($\eta = 2.5$)، نرخ یادگیری متوسط ($\eta = 0.15$) و نرخ یادگیری پایین ($\eta = 0.001$).

- **تحلیل:** رفتار هر سه منحنی را بررسی کرده و پدیده‌های **پرش از نقطه بهینه (Overshooting)** و نوسان در نرخ یادگیری بالا، و همگرایی بسیار کند در نرخ یادگیری پایین را تحلیل کنید.

۵ نحوه تحویل و بارم‌بندی

بخش پروژه	امتیاز (از ۱۰۰)
بخش ۱: پاک‌سازی داده، مدیریت مقادیر گمشده/پرت و پیش‌پردازش (بدون نشت داده)	۲۵ نمره
بخش ۲: پیاده‌سازی شبکه عصبی (انتشار جلو، تابع هزینه، منظم‌سازی و پس‌انتشار)	۴۵ نمره
بخش ۳: آزمایشگاه تحلیل (پیاده‌سازی آزمایش‌های ۳-۱ تا ۳-۵ و پاسخ‌های تحلیلی کامل)	۳۰ نمره

- فایل نوت‌بوک تکمیل‌شده خود (`ANN_Project_Student.ipynb`) را به همراه خروجی تمام سلول‌ها و نمودارها ذخیره کرده و در سامانه آموزشی بارگذاری نمایید.

- تمامی تست‌های خودکار (`assert`) موجود در نوت‌بوک باید بدون خطا اجرا شوند.